4. To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

Code –



```
  GNU nano 8.7.1
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 200
#define MAX_TOKENS 50
#define MAX_TOKEN_LENGTH 50

// Token structure
typedef struct {
    char value[MAX_TOKEN_LENGTH];
    char type[20];
} Token;

// Identify the type of token
void identifyToken(Token *token) {

    if (strcmp(token->value, "ls") == 0 ||
        strcmp(token->value, "pwd") == 0 ||
        strcmp(token->value, "cd") == 0 ||
        strcmp(token->value, "echo") == 0) {

        strcpy(token->type, "COMMAND");
    }
    else if (strcmp(token->value, "|") == 0) {
        strcpy(token->type, "PIPE");
    }
    else if (strcmp(token->value, ">") == 0 ||
             strcmp(token->value, "<") == 0) {

        strcpy(token->type, "REDIRECT");
    }
    else {
        strcpy(token->type, "ARGUMENT");
    }
}

// Split input into tokens
int tokenize(char *input, Token tokens[]) {

    int count = 0;
    int i = 0;

    while (input[i] != '\0' && count < MAX_TOKENS) {

        // Handle whitespace
        if (isspace((unsigned char)input[i])) {
            i++;
            continue;
        }

        // Identify delimiters
```



```
        // Identify delimiters
        if (input[i] == '|' ||
            input[i] == '>' ||
            input[i] == '<') {

            tokens[count].value[0] = input[i];
            tokens[count].value[1] = '\0';

            identifyToken(&tokens[count]);

            count++;
            i++;
            continue;
        }

        // Create normal token
        int j = 0;

        while (input[i] != '\0' &&
               !isspace((unsigned char)input[i]) &&
               input[i] != '|' &&
               input[i] != '>' &&
               input[i] != '<') {

            if (j < MAX_TOKEN_LENGTH - 1) {
                tokens[count].value[j++] = input[i];
            }

            i++;
        }

        tokens[count].value[j] = '\0';

        identifyToken(&tokens[count]);

        count++;
    }

    return count;
}

// Validate token stream
void validateTokens(Token tokens[], int count) {

    if (count == 0) {
        printf("Error: Empty input.\n");
        return;
    }

    // Input cannot start with pipe
    if (strcmp(tokens[0].value, "|") == 0) {
        printf("Error: Token stream cannot start with '|'.\n");
```

```c
    // Input cannot start with pipe
    if (strcmp(tokens[0].value, "|") == 0) {
        printf("Error: Token stream cannot start with '|'.\n");
        return;
    }

    // Input cannot end with pipe
    if (strcmp(tokens[count - 1].value, "|") == 0) {
        printf("Error: Token stream cannot end with '|'.\n");
        return;
    }

    // Two consecutive pipes are invalid
    for (int i = 0; i < count - 1; i++) {

        if (strcmp(tokens[i].value, "|") == 0 &&
            strcmp(tokens[i + 1].value, "|") == 0) {

            printf("Error: Consecutive pipes detected.\n");
            return;
        }
    }

    printf("Token stream is valid.\n");
}

// Display token information
void displayTokens(Token tokens[], int count) {

    printf("\nTokens:\n");
    printf("------------------------------------\n");

    for (int i = 0; i < count; i++) {

        printf("Token %d : %-15s Type: %s\n",
               i + 1,
               tokens[i].value,
               tokens[i].type);
    }

    printf("------------------------------------\n");
}

int main() {

    char input[MAX_INPUT];
    Token tokens[MAX_TOKENS];

    printf("Simple Command Tokenizer\n");
    printf("Enter a command: ");
```

```c
// Display token information
void displayTokens(Token tokens[], int count) {

    printf("\nTokens:\n");
    printf("------------------------------------\n");

    for (int i = 0; i < count; i++) {

        printf("Token %d : %-15s Type: %s\n",
               i + 1,
               tokens[i].value,
               tokens[i].type);
    }

    printf("------------------------------------\n");
}

int main() {

    char input[MAX_INPUT];
    Token tokens[MAX_TOKENS];

    printf("Simple Command Tokenizer\n");
    printf("Enter a command: ");

    fgets(input, sizeof(input), stdin);

    // Remove newline
    input[strcspn(input, "\n")] = '\0';

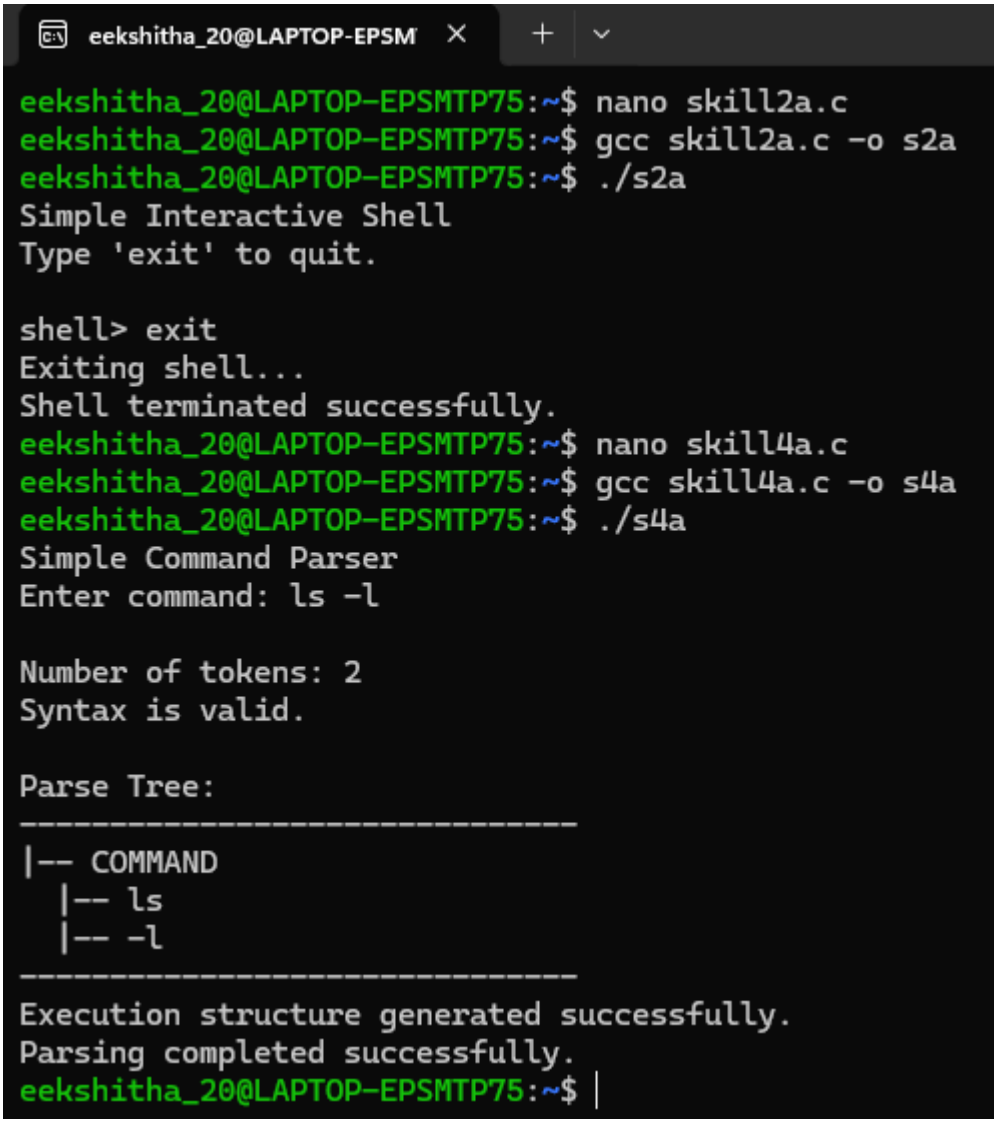
    // Split input into tokens
    int count = tokenize(input, tokens);

    // Display parsing result
    displayTokens(tokens, count);

    // Validate token stream
    validateTokens(tokens, count);

    return 0;
}
```

Result:



To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

Code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_INPUT 200
#define MAX_TOKENS 50

// Token structure
typedef struct {
    char value[50];
} Token;

// Parse tree node
typedef struct Node {
    char value[50];
    struct Node *left;
    struct Node *right;
} Node;

// Create a new parse tree node
Node *createNode(const char *value) {

    Node *newNode = (Node *)malloc(sizeof(Node));

    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        exit(1);
    }

    strcpy(newNode->value, value);
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}

// Split input into tokens
int tokenize(char *input, Token tokens[]) {

    int count = 0;

    char *token = strtok(input, " \t\n");

    while (token != NULL && count < MAX_TOKENS) {

        strcpy(tokens[count].value, token);
        count++;

        token = strtok(NULL, " \t\n");
    }

    return count;
```

```
  GNU nano 8.7.1
    return count;
}

// Validate syntax
int validateSyntax(Token tokens[], int count) {

    // Handle empty command
    if (count == 0) {
        printf("Error: Empty command.\n");
        return 0;
    }

    // Command cannot start with pipe
    if (strcmp(tokens[0].value, "|") == 0) {
        printf("Syntax Error: Command cannot start with '|'.\n");
        return 0;
    }

    // Command cannot end with pipe
    if (strcmp(tokens[count - 1].value, "|") == 0) {
        printf("Syntax Error: Command cannot end with '|'.\n");
        return 0;
    }

    // Detect consecutive pipes
    for (int i = 0; i < count - 1; i++) {

        if (strcmp(tokens[i].value, "|") == 0 &&
            strcmp(tokens[i + 1].value, "|") == 0) {

            printf("Syntax Error: Consecutive pipes detected.\n");
            return 0;
        }
    }

    printf("Syntax is valid.\n");

    return 1;
}

// Build a simple parse tree
Node *buildParseTree(Token tokens[], int count) {

    Node *root = createNode("COMMAND");
    Node *current = root;

    for (int i = 0; i < count; i++) {

        // Pipe creates a new execution structure
        if (strcmp(tokens[i].value, "|") == 0) {
```

```
GNU nano 8.7.1
        // Pipe creates a new execution structure
        if (strcmp(tokens[i].value, "|") == 0) {

            Node *pipeNode = createNode("PIPE");

            current->right = pipeNode;
            current = pipeNode;
        }

        else {

            Node *tokenNode = createNode(tokens[i].value);

            if (current->left == NULL) {
                current->left = tokenNode;
            }
            else {

                Node *temp = current->left;

                while (temp->right != NULL) {
                    temp = temp->right;
                }

                temp->right = tokenNode;
            }
        }
    }

    return root;
}

// Display parse tree
void printTree(Node *root, int level) {

    if (root == NULL)
        return;

    for (int i = 0; i < level; i++) {
        printf("    ");
    }

    printf("|-- %s\n", root->value);

    printTree(root->left, level + 1);
    printTree(root->right, level);
}

// Release parse tree memory
void freeTree(Node *root) {

|   if (root == NULL)
```

```
// Release parse tree memory
void freeTree(Node *root) {

    if (root == NULL)
        return;

    freeTree(root->left);
    freeTree(root->right);

    free(root);
}

int main() {

    char input[MAX_INPUT];
    Token tokens[MAX_TOKENS];

    printf("Simple Command Parser\n");
    printf("---------------------\n");

    printf("Enter command: ");

    fgets(input, sizeof(input), stdin);

    // Remove newline
    input[strcspn(input, "\n")] = '\0';

    // Tokenize input
    int count = tokenize(input, tokens);

    printf("\nNumber of tokens: %d\n", count);

    // Validate syntax
    if (!validateSyntax(tokens, count)) {
        return 0;
    }

    // Generate parse tree
    Node *root = buildParseTree(tokens, count);

    printf("\nParse Tree:\n");
    printf("-----------\n");

    printTree(root, 0);

    printf("-----------\n");
    printf("Execution structure generated successfully.\n");
    printf("Parsing completed successfully.\n");

    // Free allocated memory
```

```
    // Free allocated memory
    freeTree(root);

    return 0;
}
```

Result:

```
eekshitha_20@LAPTOP-EPSMTP75:~$ nano skill4b.c
eekshitha_20@LAPTOP-EPSMTP75:~$ gcc skill4b.c -o s4b
eekshitha_20@LAPTOP-EPSMTP75:~$ ./s4b
Simple Command Parser
Enter command: ls -l | grep txt

Number of tokens: 5
Syntax is valid.

Parse Tree:
------------------------------
|-- COMMAND
  |-- ls
  |-- -l
|-- PIPE
  |-- grep
  |-- txt
------------------------------
Execution structure generated successfully.
Parsing completed successfully.
eekshitha_20@LAPTOP-EPSMTP75:~$
```